

Lab 12 - Service Types: ClusterIP, NodePort and LoadBalancer

Field	Value
Lab ID	Lab 12
Day / Topic	Day 3 Services and Networking
Duration	45 minutes
Namespace	kcna-lab12
Mapping	LO3 A2 K2, K4
Cluster	Single-node kind cluster, Kubernetes v1.30 or later

1. Objective

By the end of this lab you will be able to:

1. Create and compare all four Service shapes - ClusterIP, NodePort, LoadBalancer and headless (clusterIP: None) - against a single backend.
2. Explain the superset relationship: NodePort **contains** a ClusterIP; LoadBalancer **contains** a NodePort.
3. State the default nodePort range (30000-32767) and the API-server flag that sets it, and read the exact rejection message when you go outside it.
4. Explain why type: LoadBalancer stays <pending> forever on kind and name what would have to be installed to change that.
5. Explain why a NodePort is reachable from **inside** the cluster immediately but needs kind extraPortMappings to be reachable from your laptop.
6. Show that a headless Service returns Pod IPs from DNS instead of a virtual IP.

2. Prerequisites

- Lab 11 completed - you must already be comfortable with port vs targetPort and with reading EndpointSlices.
- A running single-node kind cluster.
- Working directory:

```
cd courseware/labs/lab-12-service-types-nodeport-lb
pwd
```

Expected output (your absolute prefix will differ):

```
../../courseware/labs/lab-12-service-types-nodeport-lb
```

- Confirm you are on a kind cluster, because half this lab is about kind's

limitations:

```
kubectl get nodes -o wide
```

Expected output (one node, name starts with your kind cluster name):

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-
kind-control-plane	Ready	control-plane	52m	v1.31.0	172.18.0.2	<none>	Debian GNU/Linux 12 (bookworm)
		KERNEL-VERSION		CONTAINER-RUNTIME			
		6.10.14-linuxkit		containerd://1.7.18			

The INTERNAL-IP (172.18.0.2 here) is the address of the kind node container

on the Docker bridge network. Write yours down - Step 4 uses it.

3. Scenario

Meridian Freight's booking-edge service is the public entry point that partner freight forwarders call to lodge a booking. Three environments need three different exposure models and the platform team keeps arguing about which Service type to use:

- In-cluster callers (the quote-api from Lab 11) just need a name → ClusterIP.
- The QA team wants to hit it from a laptop against the local kind cluster →

NodePort.

- Production on a managed cloud is supposed to get a real external address →

LoadBalancer.

- A future sharded pricing engine needs to address individual Pods directly →

headless.

You will build all four against the same two Pods and record honestly which ones actually work on the training cluster and which ones cannot.

4. Step-by-step procedure

Step 1 - Namespace and backend

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab12 created
```

The backend is hashicorp/http-echo:1.0, which returns one fixed line so you can always tell it answered. That line is held in data/edge-banner.txt:

```
cat data/edge-banner.txt
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

The Deployment passes exactly this string as `-text=`:

```
kubectl apply -f manifests/10-deployment-booking-edge.yaml
kubectl rollout status deployment/booking-edge -n kcna-lab12 --timeout=120s
```

Expected output:

```
deployment.apps/booking-edge created
Waiting for deployment "booking-edge" rollout to finish: 0 of 2 updated replicas are available.
..
deployment "booking-edge" successfully rolled out
```

```
kubectl get pods -n kcna-lab12 -o wide
```

Expected output (names and IPs will differ):

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP	NODE
booking-edge-79c4b5f9d8-h4kpz	plane <none>	1/1	Running	0	25s	10.244.0.31	kind-control-
booking-edge-79c4b5f9d8-tzq7v	plane <none>	1/1	Running	0	25s	10.244.0.32	kind-control-

Start the diagnostic client now - every reachability test in this lab is run from it:

```
kubectl apply -f manifests/60-pod-net-client.yaml
kubectl wait --for=condition=Ready pod/net-client -n kcna-lab12 --timeout=60s
```

Expected output:

```
pod/net-client created
pod/net-client condition met
```

Step 2 - ClusterIP: the base case

```
kubectl apply -f manifests/20-service-clusterip.yaml
```

Expected output:

```
service/booking-edge-clusterip created
```

```
kubectl get service booking-edge-clusterip -n kcna-lab12
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-clusterip	ClusterIP	10.96.51.7	<none>	80/TCP	4s

Use it from inside the cluster:

```
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 http://booking-edge-clusterip/
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Now try it the way a beginner tries it - from your laptop. This is expected to fail; run it so you have seen the failure:

```
kubectl get service booking-edge-clusterip -n kcna-lab12 -o jsonpath='{.spec.clusterIP}{"\n"}'
```

Expected output:

```
10.96.51.7
```

```
curl --max-time 5 http://10.96.51.7/ ; echo "exit=$?"
```

Expected output:

```
curl: (28) Connection timed out after 5001 milliseconds
exit=28
```

10.96.0.0/12 is a virtual range that exists only as kube-proxy rules inside the cluster's network namespaces. Your laptop has no route to it. The supported way in is a port-forward:

```
kubectl port-forward -n kcna-lab12 service/booking-edge-clusterip 18080:80 >/tmp/pf-lab12.log
2>&1 &
sleep 2
curl -s http://127.0.0.1:18080/
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Stop the port-forward before continuing:

```
kill %1 2>/dev/null; sleep 1; echo "port-forward stopped"
```

Expected output:

```
port-forward stopped
```

Step 3 - NodePort: ClusterIP plus a port on every node

```
kubectl apply -f manifests/30-service-nodeport.yaml
```

Expected output:

```
service/booking-edge-nodeport created
```

```
kubectl get service booking-edge-nodeport -n kcna-lab12
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-nodeport	NodePort	10.96.77.140	<none>	80:30080/TCP	5s

Read PORT(S) carefully: 80:30080/TCP means *ClusterIP port 80* and *node port 30080*. A NodePort Service still has a fully working ClusterIP - prove it:

```
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 http://booking-edge-nodeport/
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Step 4 - Reach the NodePort on the node's own IP

Capture the node's internal IP into a shell variable:

```
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].address}')
echo "NODE_IP=$NODE_IP"
```

Expected output (yours will differ):

```
NODE_IP=172.18.0.2
```

Dial the node IP on port 30080 from inside the cluster. This works on stock kind - no special cluster configuration required:

```
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 "http://${NODE_IP}:30080/"
```

Expected output:

```
Meridian Freight booking-edge :: kcna-lab12
```

Now the honest part. Try the same thing from your laptop:

```
curl --max-time 5 "http://${NODE_IP}:30080/" ; echo "exit=$?"
```

Expected output on macOS or Windows (Docker Desktop runs the Docker network inside a VM, so 172.18.0.0/16 is not routable from the host):

```
curl: (28) Connection timed out after 5002 milliseconds
exit=28
```

On a Linux host with Docker running natively, the same curl usually succeeds and prints the banner, because the Docker bridge network is reachable from the host. Both results are correct for their platform - record which one you got.

The portable answer, on every platform, is to have created the kind cluster with an extraPortMappings entry. Look at the configuration that does it:

```
cat data/kind-cluster-nodeport.yaml
```

Expected output:

```
# kind cluster configuration used by README Appendix A.
...
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30080 # the Service nodePort inside the node
    hostPort: 30080      # the port on your laptop
    listenAddress: "127.0.0.1"
    protocol: TCP
```

extraPortMappings can only be set when the cluster is created. You cannot add it to a running kind cluster; you would have to create a new one (Appendix A). For today, kubectl port-forward is the correct tool and it works against any Service type.

Step 5 - LoadBalancer: the one that will not complete

```
kubectl apply -f manifests/40-service-loadbalancer.yaml
```

Expected output:

```
service/booking-edge-lb created
```

Watch it for 30 seconds. Nothing will change:

```
kubectl get service booking-edge-lb -n kcna-lab12 -w --request-timeout=30s
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-lb	LoadBalancer	10.96.184.221	<pending>	80:31544/TCP	3s
error: Timeout exceeded while reading body					

<pending> is the correct and permanent result on kind. It will never resolve.

Do not wait for it, and do not treat it as a broken lab.

Confirm there is genuinely nothing in status:

```
kubectl get service booking-edge-lb -n kcna-lab12 -o jsonpath='{.status.loadBalancer}{"\n"}'
```

Expected output:

```
{}
```

And that no controller has said anything about it:

```
kubectl describe service booking-edge-lb -n kcna-lab12 | tail -6
```

Expected output:

```
Session Affinity:      None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:                <none>
```

Why. type: LoadBalancer is a *request*, not an implementation. Something has to watch for these Services and write `.status.loadBalancer.ingress[]`:

Environment	What fulfils the request
EKS / GKE / AKS	The cloud-controller-manager, which calls the cloud's load-balancer API
Bare metal / on-prem	MetalLB (ARP/BGP), or a vendor controller
Local kind	cloud-provider-kind, a separate binary you run alongside kind
Stock kind (this cluster)	Nothing. No cloud-controller-manager is installed.

Events: <none> is the diagnostic signature: with MetalLB installed you would see IPAllocated/nodeAssigned events here; an empty event list means *nobody* is even looking at this object*.

Note what the API server did do: it allocated a ClusterIP *and* a nodePort (80:31544/TCP). That is the superset relationship - the object is a working NodePort Service that is additionally waiting for an external address that will never arrive:

```
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].address}')
LB_NODEPORT=$(kubectl get service booking-edge-lb -n kcna-lab12 -o jsonpath='{.spec.ports[0].nodePort}')
echo "auto-allocated nodePort: ${LB_NODEPORT}"
kubectl exec -n kcna-lab12 net-client -- wget -qO- --timeout=5 "http://${NODE_IP}:${LB_NODEPORT}"/"
```

Expected output (your nodePort will differ; it is allocated from 30000-32767):

```
auto-allocated nodePort: 31544
Meridian Freight booking-edge :: kcna-lab12
```

Step 6 - Headless: no virtual IP at all

```
kubectl apply -f manifests/50-service-headless.yaml
```

Expected output:

```
service/booking-edge-headless created
```

```
kubectl get service booking-edge-headless -n kcna-lab12
```

Expected output - note CLUSTER-IP is the literal None:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-headless	ClusterIP	None	<none>	80/TCP	4s

Compare the DNS answers. First the normal ClusterIP Service - one address, the virtual IP:

```
kubectl exec -n kcna-lab12 net-client -- nslookup booking-edge-clusterip
```

Expected output:

```
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: booking-edge-clusterip.kcna-lab12.svc.cluster.local
Address: 10.96.51.7
```

Now the headless Service - one A record per ready Pod:

```
kubectl exec -n kcna-lab12 net-client -- nslookup booking-edge-headless
```

Expected output (these are the Pod IPs from Step 1, not a service IP):

```
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.31
Name: booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.32
```

As in Lab 11, BusyBox may add a *** Can't find ...: No answer line for the AAAA

query. Judge by the Address: lines.

Because there is no virtual IP, there is no load balancing. The client library picks one of the returned addresses. This is exactly what a StatefulSet needs so that pod-0 and pod-1 are individually addressable (Lab 19).

Step 7 - Side-by-side comparison

```
kubectl get services -n kcna-lab12 -o wide
```

Expected output:

NAME SELECTOR	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-clusterip app=booking-edge	ClusterIP	10.96.51.7	<none>	80/TCP	6m
booking-edge-headless app=booking-edge	ClusterIP	None	<none>	80/TCP	1m
booking-edge-lb app=booking-edge	LoadBalancer	10.96.184.221	<pending>	80:31544/TCP	3m
booking-edge-nodeport app=booking-edge	NodePort	10.96.77.140	<none>	80:30080/TCP	5m

Four Services, one set of Pods, one identical selector. The Service type changes only *how the address is exposed*, never *what is behind it*. Confirm that all four share the same backends:

```
kubectl get endpointslices -n kcna-lab12 \
-o custom-columns='SLICE:.metadata.name,SERVICE:.metadata.labels.kubernetes\.io/service-name,
PORTS:.ports[*].port,ENDPOINTS:.endpoints[*].addresses[0]'
```

Expected output:

SLICE	SERVICE	PORTS	ENDPOINTS
booking-edge-clusterip-9xnvp	booking-edge-clusterip	5678	10.244.0.31,10.244.0.32
booking-edge-headless-2rk4d	booking-edge-headless	5678	10.244.0.31,10.244.0.32
booking-edge-lb-lm7ct	booking-edge-lb	5678	10.244.0.31,10.244.0.32
booking-edge-nodeport-vv8sq	booking-edge-nodeport	5678	10.244.0.31,10.244.0.32

The recorded expectations for each type live in data/service-matrix.csv:

```
cat data/service-matrix.csv
```

Expected output:

```
name,type,cluster_ip,port,target_port,node_port,external_ip_on_kind,reachable_from_laptop
booking-edge-clusterip,ClusterIP,allocated,80,5678,none,none,no - use kubectl port-forward
booking-edge-nodeport,NodePort,allocated,80,5678,30080,none,only if the kind cluster was
created with extraPortMappings
booking-edge-lb,LoadBalancer,allocated,80,5678,auto,pending-forever,no - kind has no cloud-
controller-manager
booking-edge-headless,ClusterIP,None,80,5678,none,none,no - DNS returns Pod IPs and there is
no virtual IP
```

verification/checks.sh asserts the live cluster against every row of this file - including asserting that the LoadBalancer is pending.

5. Verification

```
bash verification/checks.sh
```

Expected output:

```

== Lab 12 verification: Service types on kind ==
PASS namespace kcna-lab12 exists
PASS deployment booking-edge has 2/2 ready replicas
PASS matrix row booking-edge-clusterip: type=ClusterIP port=80 targetPort=5678
PASS matrix row booking-edge-clusterip: clusterIP allocated as expected
PASS matrix row booking-edge-clusterip: no nodePort allocated, as expected
PASS matrix row booking-edge-nodeport: type=NodePort port=80 targetPort=5678
PASS matrix row booking-edge-nodeport: clusterIP allocated as expected
PASS matrix row booking-edge-nodeport: nodePort is 30080
PASS matrix row booking-edge-nodeport: nodePort 30080 is inside 30000-32767
PASS matrix row booking-edge-lb: type=LoadBalancer port=80 targetPort=5678
PASS matrix row booking-edge-lb: clusterIP allocated as expected
PASS matrix row booking-edge-lb: auto-allocated nodePort 31544 is inside 30000-32767
PASS EXPECTED ON kind: booking-edge-lb EXTERNAL-IP is still <pending> (no cloud-controller-
manager)
PASS matrix row booking-edge-headless: type=ClusterIP port=80 targetPort=5678
PASS matrix row booking-edge-headless: clusterIP is None as expected
PASS matrix row booking-edge-headless: no nodePort allocated, as expected
PASS all four Services resolve to the same 2 endpoint addresses
PASS ClusterIP Service serves the exact text in data/edge-banner.txt
PASS NodePort 30080 answers on the node InternalIP from inside the cluster
PASS headless Service DNS returns Pod IPs, not a virtual IP
-- 20 passed, 0 failed --

```

See verification/expected-output.md for the annotated reference.

6. Failure injection - a nodePort outside the allowed range

6.1 Break it

```
kubectl apply -f manifests/70-service-nodeport-invalid.yaml
```

Expected output (this is a rejection, and the command exits non-zero):

```
The Service "booking-edge-nodeport-invalid" is invalid: spec.ports[0].nodePort: Invalid value:
8080: provided port is not in the valid range. The range of valid ports is 30000-32767
```

6.2 Diagnose

Three facts are packed into that one line:

1. The object was never created. Confirm:

```
kubectl get service booking-edge-nodeport-invalid -n kcna-lab12
```

Expected output:

```
Error from server (NotFound): services "booking-edge-nodeport-invalid" not found
```

2. The rejection came from the API server, not from your YAML tooling. The file is schema-valid - kubeconform accepts it, because "is 8080 inside the node-port range" is *API-server validation*, not OpenAPI schema validation. This is the difference between a manifest that is *well-formed* and one that is *admissible*.
3. The range is a cluster setting, not a constant. It comes from the API server

flag --service-node-port-range, default 30000-32767. Read it off this cluster's control plane:

```
kubectl get pod -n kube-system -l component=kube-apiserver \
  -o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep -i 'node-port' ||
echo "flag not set - cluster is using the default 30000-32767"
```

Expected output on stock kind (the flag is not passed, so the default applies):

```
flag not set - cluster is using the default 30000-32767
```

6.3 Fix and re-verify

```
kubectl apply -f manifests/70-service-nodeport-invalid.yaml --dry-run=client -o yaml \
  | sed 's/nodePort: 8080/nodePort: 30081/' \
  | kubectl apply -f -
```

Expected output:

```
service/booking-edge-nodeport-invalid created
```

```
kubectl get service booking-edge-nodeport-invalid -n kcna-lab12
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-nodeport-invalid	NodePort	10.96.9.201	<none>	80:30081/TCP	4s

6.4 Second failure to be aware of - port already allocated

Try to claim 30080 a second time:

```
kubectl create service nodeport clash --tcp=80:5678 --node-port=30080 -n kcna-lab12
```

Expected output:

```
The Service "clash" is invalid: spec.ports[0].nodePort: Invalid value: 30080: provided port is
already allocated
```

Node ports are a cluster-wide shared resource. Hard-coding them, as this lab does for teaching purposes, does not scale past a handful of Services - which is exactly the argument for Ingress (Lab 13).

7. Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
EXTERNAL-IP stuck at <pending>	No load-balancer implementation in the cluster	kubectl describe svc <name> → Events: <none>	Expected on kind. Use NodePort or port-forward; install cloud-provider-kind or MetalLB if you truly need it
curl http://<clusterIP>/ from the laptop times out	ClusterIP range is virtual and cluster-internal	curl exits 28; the same URL works from net-client	kubectl port-forward svc/<name> 18080:80
curl http://<nodeIP>:30080/ times out from a macOS/Windows laptop	Docker's bridge network is inside a VM	Same request from net-client succeeds	Recreate kind with extraPortMappings (Appendix A) or use port-forward
provided port is not in the valid range	nodePort outside --service-node-port-range	Read the error; check the flag on kube-apiserver	Choose a port in 30000-32767, or omit nodePort and let the API server allocate
provided port is already allocated	Another Service in any namespace holds that nodePort	kubectl get svc -A -o jsonpath='{range .items[*]}{.spec.ports[*].nodePort}{"\n"}{end}'	Pick a free port or omit nodePort
Headless Service returns No answer	Headless DNS only publishes ready endpoints; zero ready Pods means zero records	kubectl get endpointslices -n kcna-lab12 -l kubernetes.io/service-name=booking-edge-headless	Fix Pod readiness first
ImagePullBackOff on hashicorp/http-echo:1.0	No registry access from the cluster	kubectl describe pod <pod> -n kcna-lab12 → Failed to pull image	Substitute nginx:1.27-alpine with targetPort: 80 in all four Services; the Service-type behaviour is unchanged

8. Cleanup

Delete only this lab's namespace:

```
kubectl delete namespace kcna-lab12
```

Expected output:

```
namespace "kcna-lab12" deleted
```

If you left a port-forward running, stop it:

```
kill %1 2>/dev/null; echo "done"
```

Expected output:

```
done
```

Never run `kubectl delete svc --all` without `-n kcna-lab12`.

9. What you learned

- The four shapes are nested, not parallel: ClusterIP □ NodePort □ LoadBalancer;

headless is ClusterIP with the VIP deliberately removed.

- The Service type only changes the front door. All four of your Services produced

identical EndpointSlices from the identical selector.

- nodePort comes from a cluster-wide pool, default 30000-32767

(--service-node-port-range), and collides across namespaces.

- On kind, type: LoadBalancer never leaves <pending>, because no

cloud-controller-manager or MetalLB is installed. Events: <none> on the Service is the proof that no controller is watching it.

- A NodePort works *inside* the cluster on stock kind; reaching it from the host needs

extraPortMappings chosen at cluster-creation time (or a Linux host with native Docker).

- clusterIP: None removes the virtual IP; DNS then returns one A record per ready

Pod and the client chooses.

- API-server validation (nodePort range, port already allocated) is stricter than

schema validation - a file kubeconform accepts can still be rejected on apply.

10. Further reading

- Service, and the publishing types - <https://kubernetes.io/docs/concepts/services-networking/service/#publishing-services-service-types>
- Headless Services - <https://kubernetes.io/docs/concepts/services-networking/service/#headless-services>
- Virtual IPs and Service proxies - <https://kubernetes.io/docs/reference/networking/virtual-ips/>
- kube-apiserver flags (--service-node-port-range) - <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>
- Cloud controller manager - <https://kubernetes.io/docs/concepts/architecture/cloud-controller/>
- Use Port Forwarding to Access Applications in a Cluster - <https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

Appendix A - a kind cluster whose NodePort is reachable from the host

Optional, and it creates a second cluster. Only do this if you want to see <http://localhost:30080> work. It does not delete your existing cluster.

```
kind create cluster --name kcna-nodeport --config data/kind-cluster-nodeport.yaml
```

Expected output (abridged):

```
Creating cluster "kcna-nodeport" ...  
✓ Ensuring node image (kindest/node:v1.31.0) ■  
✓ Preparing nodes ■  
✓ Writing configuration ■  
✓ Starting control-plane ■■  
✓ Installing CNI ■  
✓ Installing StorageClass ■  
Set kubectl context to "kind-kcna-nodeport"
```

Redeploy this lab into that cluster and `curl http://127.0.0.1:30080/` will return the banner, because Docker is publishing the node container's port 30080 on your loopback interface. `type: LoadBalancer` still stays <pending> there - `extraPortMappings` solves NodePort host access, not the missing cloud provider.

When finished, switch back to your training cluster:

```
kubectl config use-context kind-kind
```